Primo 1: Etiquetas de precio del Baratie (hermano del Ejercicio 1)

Qué se simula: una batería de pruebas de caja negra con arrays paralelos de entradas y resultados esperados.

Enunciado: el cocinero Sanji quiere etiquetar los platos del restaurante Baratie según su precio en berries. Escribe el método `static String etiquetaPrecio(int berries)` con estas reglas, que se evalúan en este orden:

- Si `berries` es menor que 0, devuelve `"ERROR"`.
- Si es menor que 50, devuelve `"BARATO"`.
- Si es menor que 200, devuelve `"NORMAL"`.
- En cualquier otro caso, devuelve `"LUJO"`.

En `main`, declara estos dos arrays:

- `int[] entradas = {-5, 0, 49, 50, 199, 200};`
- `String[] esperados = {"ERROR", "BARATO", "BARATO", "NORMAL", "NORMAL", "LUJO"};`

Recorre los índices con un `for`. En cada vuelta llama a `etiquetaPrecio` con la entrada, guarda el resultado en `obtenido` y compáralo con el esperado usando `.equals`. Imprime una línea por caso con este formato, donde el número de caso empieza en 1:

`Caso N: entrada=X esperado=Y obtenido=Z -> OK` (o `FALLO` si no coinciden)

No imprimas nada más. Se espera que todos los casos den `OK`.

text

```
Caso 1: entrada=-5 esperado=ERROR obtenido=ERROR -> OK
Caso 2: entrada=0 esperado=BARATO obtenido=BARATO -> OK
Caso 3: entrada=49 esperado=BARATO obtenido=BARATO -> OK
Caso 4: entrada=50 esperado=NORMAL obtenido=NORMAL -> OK
Caso 5: entrada=199 esperado=NORMAL obtenido=NORMAL -> OK
Caso 6: entrada=200 esperado=LUJO obtenido=LUJO -> OK
```

Primo 2: Ingresos del banco de Nami (hermano del Ejercicio 2)

Qué se simula: un mini framework de pruebas unitarias automáticas con contadores `static`.

Enunciado: la navegante Nami guarda sus ingresos en un array de enteros (berries). Escribe un programa con estas piezas:

- Dos variables `static int` llamadas `pruebasTotales` y `pruebasFallidas`, ambas iniciadas en 0.
- El método `static int sumarIngresos(int[] ingresos)`, que devuelve la suma de todos los elementos con un bucle `for`. Si el array está vacío, devuelve 0.
- El método `static void comprobar(String nombre, int esperado, int obtenido)`. Suma 1 a `pruebasTotales`. Si `esperado == obtenido`, imprime `[PASA] ` (con dos espacios tras el corchete) seguido del nombre. Si no, suma 1 a `pruebasFallidas` e imprime `[FALLA]` (con un espacio tras el corchete) seguido del nombre, `esperado=` y el valor esperado, `obtenido=` y el valor obtenido.

En `main`, ejecuta estas cuatro pruebas en este orden:

1. `"cuatro ingresos"`, esperado 100, con el array `{10, 20, 30, 40}`.
2. `"un ingreso"`, esperado 7, con el array `{7}`.
3. `"sin ingresos"`, esperado 0, con un array vacío (`new int[]{}`).
4. `"prueba erronea a proposito"`, esperado 20, con el array `{8, 9}`. Fallará a propósito.

Al terminar, imprime `Total: ` seguido de `pruebasTotales`, `Fallidas: ` y `pruebasFallidas`.

text

```
[PASA]  cuatro ingresos
[PASA]  un ingreso
[PASA]  sin ingresos
[FALLA] prueba erronea a proposito esperado=20 obtenido=17
Total: 4 Fallidas: 1
```

Primo 3: Entrenamiento de Zoro con trazas (hermano del Ejercicio 3)

Qué se simula: el depurador (breakpoint condicional, watch y modificación de variable) con `println` y un `if`.

Enunciado: Zoro entrena con pesas durante cinco días. Las horas de cada día están en `int[] horas = {4, 6, 3, 7, 2};`. Declara `int total = 0;` como variable a vigilar. Recorre el array con un `for` cuya variable se llame `dia` (de 0 a 4). En cada vuelta haz esto, en este orden:

1. Imprime `[TRAZA] dia=` + `dia` + `antes total=` + `total`.
2. Suma `horas[dia]` a `total`.
3. Imprime `[TRAZA] dia=` + `dia` + `despues total=` + `total`.

4. Si `total > 12`, simula un punto de ruptura. Imprime `[BREAKPOINT] total=` + `total` + `supera 12 en dia` + `dia`. Asigna `total = 12`. Imprime `[WATCH] total corregido a` + `total`.

Al acabar el bucle, imprime `Total final:` seguido de `total`.

text

```
[TRAZA] dia=0 antes total=0
[TRAZA] dia=0 despues total=4
[TRAZA] dia=1 antes total=4
[TRAZA] dia=1 despues total=10
[TRAZA] dia=2 antes total=10
[TRAZA] dia=2 despues total=13
[BREAKPOINT] total=13 supera 12 en dia 2
[WATCH] total corregido a 12
[TRAZA] dia=3 antes total=12
[TRAZA] dia=3 despues total=19
[BREAKPOINT] total=19 supera 12 en dia 3
[WATCH] total corregido a 12
[TRAZA] dia=4 antes total=12
[TRAZA] dia=4 despues total=14
[BREAKPOINT] total=14 supera 12 en dia 4
[WATCH] total corregido a 12
Total final: 12
```

PROMETEO
by thePower

Primo 4: Tesoros de Nami y registro de incidencias (hermano del Ejercicio 4)

Qué se simula: un informe de incidencias con un contador y un formato fijo.

Enunciado: hay que contar cuántos tesoros valen al menos un mínimo. Escribe estos elementos:

- `static int contarRicosErroneo(int[] monedas, int minimo)`. Usa un contador que empieza en 0 y un `for` con la condición `i < monedas.length - 1` (el error intencionado). Suma 1 al contador cuando `monedas[i] >= minimo`. Devuelve el contador.
- `static int contarRicos(int[] monedas, int minimo)`. Es igual, pero con la condición correcta `i < monedas.length`.
- Una variable `static int numIncidencia = 0;`.

- `static void verificar(String funcion, int esperado, int obtenido)`. Solo si `esperado != obtenido`, suma 1 a `numIncidencia` e imprime `INC-` + `numIncidencia` + `| Funcion: ` + `funcion` + `| Esperado: ` + `esperado` + `| Obtenido: ` + `obtenido`. Si coinciden, no imprime nada.

En `main`, declara `int[] botin = {120, 30, 80, 200, 90};` y haz estas tres verificaciones en este orden:

1. `verificar("contarRicosErroneo", 4, contarRicosErroneo(botin, 80));`
2. `verificar("contarRicos", 4, contarRicos(botin, 80));`
3. `verificar("contarRicosErroneo (vacio)", 0, contarRicosErroneo(new int[]{}, 80));`

Al final, imprime `Incidencias detectadas: ` seguido de `numIncidencia`.

text

```
INC-1 | Funcion: contarRicosErroneo | Esperado: 4 | Obtenido: 3
Incidencias detectadas: 1
```

Primo 5: Daño de Zoro con dado falso (hermano del Ejercicio 5)

Qué se simula: un doble de prueba (stub), con valores fijos que sustituyen al dado real.

Enunciado: el daño del ataque de Zoro depende de un dado. Escribe estos métodos:

- `static int lanzarDado()`, que devuelve `(int) (Math.random() * 6) + 1`. Es la fuente real y no determinista. **No la llames desde `main`.**
- `static int danoZoro(int dado)`. Si `dado < 1 || dado > 6`, devuelve -1. En otro caso, devuelve `dado * 15`.

En `main`, declara estos dos arrays:

- `int[] dadosFalsos = {2, 5, -3, 9};`
- `int[] esperados = {30, 75, -1, -1};`

Recorre los índices con un `for`. En cada vuelta calcula `dano = danoZoro(dadosFalsos[i])`. Guarda en `estado` el texto `"OK"` si `dano == esperados[i]` o `"FALLO"` en otro caso. Imprime esta línea:

```
Dado falso=X dano=Y -> Z
```

text

```
Dado falso=2 dano=30 -> OK
```

```
Dado falso=5 dano=75 -> OK
Dado falso=-3 dano=-1 -> OK
Dado falso=9 dano=-1 -> OK
```

Tabla de primos

Primo	Hermano de	Qué cambia	Qué se mantiene
1. Etiquetas de precio	Ejercicio 1	Dominio (restaurante), tramos (50/200), datos y etiquetas	Cuatro ramas `if`, arrays paralelos, bucle `for`, valores frontera
2. Ingresos de Nami	Ejercicio 2	Dominio (banco), datos y nombres (`sumarIngresos`)	`comprobar`, contadores `static`, caso de array vacío, prueba fallida a propósito
3. Entrenamiento de Zoro	Ejercicio 3	Dominio (entrenamiento), datos, límite (12) y mensajes	Trazas antes y después, breakpoint condicional, corrección de la variable
4. Tesoros y registro	Ejercicio 4	Dominio (tesoros), datos, mínimo (80) y nombres	Error `length - 1`, `verificar` con `numIncidencia`, formato `INC-N`
5. Daño con dado falso	Ejercicio 5	Daño (×15), dados falsos `{2, 5, -3, 9}` y nombres	Stub por parámetro, validación 1–6, `lanzarDado` sin usa

PROMETEO by thePower